Faculty of Computer Science and Artificial Intelligence, Capital University

Subject: Software Engineering – 2

Software Requirements Specification for

“Online Art Auction Platform”

Table of Contents

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to provide a detailed description of the functional and non-functional requirements for the Online Art Auction Platform. This system is designed to enable artists to showcase their artworks and allow buyers to participate in auctions by placing bids in real time. The document defines system behavior, features, constraints, and interfaces to ensure a clear understanding between stakeholders, developers, and evaluators throughout the development lifecycle.

1.2 Document Conventions

This document follows standard software engineering conventions to ensure clarity and consistency. Key system roles such as Admin, Artist, Buyer, Guest User, and System are capitalized throughout the document. Functional requirements are numbered using a structured format such as 3.1-1, where the first two digits represent the main feature category and the final digit represents the specific requirement within that feature. The structure of this document follows an IEEE-style SRS format to maintain professional documentation standards.

1.3 Intended Audience and Reading Suggestions

This SRS is intended for developers, system designers, testers, project supervisors, and academic evaluators involved in the project. Developers should focus on system features and technical constraints, while testers should refer to functional requirements for validation. It is recommended that readers follow the document sequentially, beginning with the introduction, then the overall description, system features, interface requirements, and non-functional requirements.

1.4 Project Scope

The Online Art Auction Platform is a web-based system that facilitates the interaction between artists and buyers in a digital auction environment. The platform allows artists to upload and manage artworks, while buyers can browse available items and place bids during active auction periods. The system incorporates real-time bidding, ensuring immediate updates and competitive interaction among users. Additionally, the platform includes an administrative layer responsible for approving artists and moderating content to maintain quality and security. The system also supports email notifications and a simplified payment submission process for winning bidders.

2. Overall Description

2.1 Product Perspective

The Online Art Auction Platform is a standalone web application built using modern web technologies, including React with Vite for the frontend and Spring Boot for the backend. The system follows a microservice-based architecture, where major business functions such as authentication, artwork management, bidding, notifications, and administration are separated into independent services. These services communicate through APIs and Kafka topics

2.2 Product Features

The system provides a comprehensive set of features that support the complete lifecycle of an online auction. Users can register and authenticate based on their roles, with different permissions assigned to Admins, Artists, and Buyers. Artists can upload artworks with detailed information and manage them through CRUD operations, while Admins review and approve both artist accounts and artwork submissions. Buyers can browse artworks, apply filters such as artist name, category, or tags, and participate in real-time bidding. The system ensures fair bidding by enforcing a minimum increment rule and provides transparency through bid history visibility. Additionally, buyers can maintain a watchlist to track auctions of interest, and the system automatically determines auction winners and notifies them via email upon completion.

2.3 User Classes and Characteristics

The system consists of several user classes, each with distinct roles and responsibilities. Guest users can access the platform without authentication, allowing them to browse artworks and view auction details, but they are restricted from placing bids. Buyers are registered users who can actively participate in auctions, place bids, manage watchlists, and view bid histories. Artists are responsible for uploading and managing artworks, controlling auction durations, and ensuring their listings meet platform standards. Admin users have the highest level of access, allowing them to approve or reject artist registrations and artwork submissions, ensuring system integrity and content quality. Each user class interacts with the system differently, requiring tailored interfaces and permissions.

2.4 Operating Environment

The system operates within a web-based environment and is accessible through modern browsers such as Google Chrome, Mozilla Firefox, and Microsoft Edge. The backend runs on a server environment capable of supporting Spring Boot applications, while the frontend is delivered through a React-based interface. The services are deployed using Docker containers to provide isolated and consistent runtime environments for frontend, backend services, databases, and supporting services.

2.5 Design and Implementation Constraints

The system must adhere to several technical and operational constraints. It must be developed using the specified technologies, namely React with Vite for the frontend and Spring Boot for the backend. Security requirements mandate proper authentication, authorization, and protection of user data, including secure handling of login credentials. Additionally, the system must enforce bidding rules, such as minimum bid increments, and ensure data consistency during concurrent bidding scenarios. The system must also follow a microservice-based architecture, and each major service should be containerized using Docker to simplify deployment and service management.

3. System Features

3.1 User Authentication

Description

The system provides authentication mechanisms that allow users to register, log in, and log out securely. While guest users can browse artworks without authentication, buyers must log in to participate in auctions and place bids.

Functional Requirements

- 3.1-1: The system must allow different user types to log in and log out.
- 3.1-2: The system must allow guest users to browse artworks without authentication.
- 3.1-3: The system must prevent guest users from placing bids.
- 3.1-4: Buyers must be authenticated before placing bids.
- 3.1-5: The system must assign permissions based on user role.

3.2 Artist Account Approval

Description

The system allows users to register as Artists, but newly registered Artist accounts must be reviewed by an Admin before they are fully accepted. This approval process helps protect the platform from fake accounts and ensures that only valid artists can publish artworks for auction.

Functional Requirements

- 3.2-1: The system must allow users to submit Artist registration requests.
- 3.2-2: Admins must be able to accept newly registered Artist accounts.
- 3.2-3: Admins must be able to reject newly registered Artist accounts.
- 3.2-4: The system must prevent pending or rejected Artists from publishing visible artworks.

3.3 Artwork Management

Description

Artists are provided with tools to create, update, list, and delete their artworks. Each artwork must contain all required auction and artwork details, and images are uploaded directly to the platform.

Functional Requirements

- 3.3-1: Artists must be able to create artwork posts.
- 3.3-2: Artists must be able to list their submitted artworks.
- 3.3-3: Artists must be able to update artwork details.
- 3.3-4: Artists must be able to delete artwork posts.
- 3.3-5: Each artwork must include Artist Name, Title, Description, Initial Price, Auction Start Time, Auction End Time, Category, Tags, and Artwork Image.
- 3.3-6: The system must allow Artists to upload artwork images directly.

3.4 Artwork Approval

Description

Artwork posts submitted by Artists must be reviewed by an Admin before they become visible to Buyers. This moderation process ensures that published artworks are appropriate, complete, and aligned with platform standards.

Functional Requirements

- 3.4-1: Admins must be able to list pending artwork posts.
- 3.4-2: Admins must be able to accept artwork posts.
- 3.4-3: Admins must be able to reject artwork posts.
- 3.4-4: Accepted artworks must become visible to buyers.

- 3.4-5: Rejected artworks are deleted and not presented for public browsing.

3.5 Browsing and Filtering Artworks

Description
Buyers and guest visitors can browse available artworks through the platform. The browsing feature helps users discover auctions and search for artworks that match their interests. Filtering options improve the user experience by allowing users to narrow results based on artist name, category, or tags.

Functional Requirements
- 3.5-1: The system must allow visitors and Buyers to browse visible artworks.
- 3.5-2: The system must allow users to filter artworks by artist name.
- 3.5-3: The system must allow users to filter artworks by category.
- 3.5-4: The system must allow users to filter artworks by tags.
- 3.5-5: The system must display artwork details clearly on the artwork details page.

3.6 Real-Time Auction and Bidding

Description
The platform supports real-time auctions where buyers can place bids during a defined auction period. The system uses WebSocket communication to update bids instantly across all users viewing the same auction. The system validates every bid to ensure fairness and prevent invalid bidding behavior.

Functional Requirements
- 3.6-1: Buyers must be able to place bids during the auction period.
- 3.6-2: The system must reject bids submitted before the auction start time.
- 3.6-3: The system must reject bids submitted after the auction end time.
- 3.6-4: Each new bid must be at least $10 higher than the previous bid.
- 3.6-5: The system must update the current highest bid in real time.

3.7 Bid History

Description
The system provides transparency by allowing users to view the full history of bids placed on an artwork, including relevant details. Any visitor can view the bid history including bidder name, price, and timestamp.

Functional Requirements
- 3.7-1: The system must display bid history for each artwork.
- 3.7-2: Bid history must include bidder name, bid amount, and timestamp.
- 3.7-3: The system must allow visitors to view bid history without logging in.

3.8 Watchlist System

Description
Buyers can add artworks to a watchlist to monitor ongoing auctions and receive updates. This helps Buyers follow multiple auctions without searching for the same artworks repeatedly.

Functional Requirements

- 3.8-1: Buyers must be able to add artworks to a watchlist.
- 3.8-2: Buyers must be able to remove artworks from a watchlist.
- 3.8-3: The system must display watched artworks in the Buyer account area.
- 3.8-4: The system must display the current status of watched auctions.

3.9 Auction Control and Extension

Description

Artists have the ability to manage the timing of their auctions and extend deadlines when necessary. This feature allows Artists to adjust auction duration in special cases, and the system must update the auction status accordingly.

Functional Requirements

- 3.9-1: Artists must be able to extend auction end time.
- 3.9-2: The system must update the auction end time after extension.
- 3.9-3: The system must display the updated auction end time to users.

3.10 Winner Determination and Payment Submission

Description

After the auction ends, the system automatically determines the winning bidder based on the highest valid bid. The winner is then guided to submit purchase details through a payment submission form. The form represents payment submission only and does not perform full payment gateway processing in the current version.

Functional Requirements

- 3.10-1: The system must automatically determine the winning bidder after the auction ends.
- 3.10-2: The highest valid bidder must be selected as the winner.
- 3.10-3: The system must store the auction result.
- 3.10-4: The system must provide a payment submission form for the winning buyer.
- 3.10-5: The system must not declare a winner if no valid bids were placed.

3.11 Email Notifications

Description

The system sends notifications to users regarding important auction events, especially when a buyer wins an auction. The email notification includes purchase details such as artwork title, final bid amount, and next steps for payment submission.

Functional Requirements

- 3.11-1: The system must send an email notification to the winning Buyer after the auction ends.
- 3.11-2: The notification must include artwork details and final purchase amount.
- 3.11-3: The notification must include instructions for completing the payment submission form.

3.12 Admin Management Tools

Description

Admin users are responsible for maintaining system quality by reviewing and approving content and users. They can review Artist registrations and artwork submissions, then accept or reject them. Admin tools are essential for preventing inappropriate content and ensuring that only approved Artists and artworks are available on the platform.

Functional Requirements

- 3.12-1: Admins must be able to view pending Artist accounts.
- 3.12-2: Admins must be able to approve or reject Artist accounts.
- 3.12-3: Admins must be able to view pending artwork submissions.
- 3.12-4: Admins must be able to approve or reject artwork submissions.
- 3.12-5: Admin pages must require authentication and Admin authorization.

4. External Interface Requirements

4.1 User Interfaces

The platform provides a responsive and user-friendly interface that supports different user roles. The interface includes a homepage displaying active auctions, detailed artwork pages with bidding options and bid history, user dashboards for buyers and artists, and an admin panel for system management. The design ensures ease of navigation and accessibility across devices. Forms should provide validation messages when required fields are missing or invalid. Important actions such as bidding, deleting artwork, approving submissions, and extending auctions should be presented clearly to reduce user mistakes.

4.2 Hardware Interfaces

The platform interacts with standard user devices such as desktops and laptops. Input methods include mouse clicks, keyboard input, and touchscreen interactions. The system also supports image uploading from user devices, allowing Artists to upload artwork images from local storage.

4.3 Software Interfaces

The system integrates with several software components, including a backend built using Spring Boot and a frontend developed using React with Vite. Data exchange between components is handled using JSON format. The system also requires a database to store users, artworks, bids, watchlists, approvals, auction results, and payment submission information.

4.4 Communication Interfaces

The system communicates over HTTP/HTTPS protocols for standard requests and uses WebSocket connections for real-time bidding updates. Email protocols are used for sending notifications, and all data transmissions should be secured to ensure user privacy and system integrity. Real-time bid messages should include necessary information such as artwork ID, bid amount, bidder name, and timestamp while avoiding exposure of unnecessary sensitive user data.

5. Non-Functional Requirements

5.1 Performance Requirements

The system should respond quickly to user actions such as browsing artworks, filtering results, logging in, and submitting forms. Real-time bid updates should be delivered with minimal delay so that buyers can follow auctions accurately. The backend should be able to handle multiple users viewing and bidding on the same artwork without losing data consistency.

5.2 Security Requirements

The system must protect user accounts and restrict access according to user roles. Buyers, Artists, and Admins should only access features permitted for their roles. Passwords must be stored securely, and admin pages must require authentication and authorization. The system must validate all bids and uploaded data to prevent invalid input or unauthorized changes.

5.3 Reliability Requirements

The system should maintain correct auction results even when multiple bids are placed close together. If a bid is accepted, it must be stored accurately and reflected in the current highest bid. The system should also preserve auction history and winner information after auctions end.

5.4 Usability Requirements

The user interface should be easy to understand for visitors, buyers, artists, and admins. Users should be able to browse artworks, place bids, and manage listings without needing advanced technical knowledge. Error messages should be clear and displayed near the relevant fields.

5.5 Maintainability Requirements

The system should be developed using a clear structure that separates frontend components, backend services, controllers, models, and database operations. This structure makes the system easier to maintain, test, and extend in the future.

5.6 Portability Requirements

The platform should operate on common modern browsers and support different screen sizes. The React frontend should be responsive, and the Spring Boot backend should be deployable on standard server environments that support Java applications.